Software-ingeniaritza agentikoa

Lantalde hibridoak kudeatzeko eskuliburua

Inigo Lopez-Gazpio

Universidad del País Vasco
Euskal Herriko Unibertsitatea

© Udako Euskal Unibertsitatea
© Euskal Herriko Unibertsitatea
© Inigo Lopez-Gazpio

UEUren ISBNa: [betetzeko]
UPV/EHUren ISBNa: [betetzeko]
Lege-gordailua: [betetzeko]

Inprimategia: [betetzeko]
Azalaren diseinua: [betetzeko]
Hizkuntza-zuzenketen arduraduna: [betetzeko]

Banatzaileak: UEU. Erribera 14, 1, D BILBO. Telf. 946790546
Helbide elektronikoa: argitalpenak@ueu.eus
www.ueu.eus

Euskal Herriko Unibertsitatea
Helbide elektronikoa: argitalpenak@ehu.eus
www.ehu.eus/argitalpenak

Galarazita dago liburu honen kopia egitea, osoa nahiz zatikakoa, edozein modutara delarik ere, edizio honen Copyright-jabeen baimenik gabe.

Esker ona Udako Euskal Unibertsitateari eta Euskal Herriko Unibertsitateari, euskarazko ikasmaterialgintza sustatzeko egiten duten lanagatik.

Gaien Aurkibidea

1 Hitzaurrea eta helburuak 11

1.1 Nondik gatozen, non gauden 13

1.2 Zer ikasiko dugu 14

1.3 Helburuak 15

1.4 Eskerrak 16

2 Taldeak eta rolak 17

2.1 Belbinen talde-rolak 18

2.2 *Scrum*eko rolak: produktu-jabea, *Scrum* zuzendaria, garatzaileak 18

2.3 Agenteak taldean: zer rol bai, zer rol ez 18

2.4 Ariketa: taldearen rol-mapa, agenteak barne 18

3 Erabiltzaile-historioak eta zereginak 19

3.1 Erabiltzaile-historioak eta *INVEST*, agenteen ikuspegitik 21

3.2 Onarpen-irizpideak eta egindakoaren definizioa 21

3.3 Zeregina kontratu gisa: *issue* baten anatomia 21

3.4 UML arina: klase-, sekuentzia-, erabilera-kasu- eta egoera-makinaren diagramak 21

4 Biltegia, lan-eremu partekatua 23

4.1 Urruneko helbidea: Git teknologia, GitHub plataforma 25

4.2 Zereginaren araberako adarrak 25

4.3 *Pull request*-a berrikuspen-unitate gisa 25

4.4 Lerro nagusiaren babes-arauak 25

4.5 Integrazio jarraitua epaile gisa 25

5 Ariketak: zereginak eta taulak GitHuben 27

5.1 Ariketa ebatziak 27

5.2 Proposatutako ariketa gehigarriak 31

6 Kodetzeko agenteak 33

6.1 Zer da kodetzeko agente bat: eredua, tresnak eta begizta 34

6.2 Testuingurua, baimenak, isolamendua eta kostua 34

6.3 Agenteak biltegiarekin eta proiektu-taularekin hizketan 34

6.4 Huts egiteko moduak 34

7 Scrumban agentikoa 35

7.1 Nondik dator terminoa? 35

7.2 *Scrum*: inplementazioaren erritmoa 36

7.3 *Kanban*: exekuzioaren fluxua 37

7.4 Zergatik nahastu biak lantalde hibrido batean? 38

7.5 Scrumban agentikoa, begirada batean 40

7.6 Zereginaren egoera-makina 41

7.7 Eskalatze-protokoloa 41

7.8 Trazabilitatea: zereginetik kodera 41

8 Ariketak: kaixo, Scrumban agentikoa! 43

8.1 Ariketa ebatziak 43

8.2 Proposatutako ariketa gehigarriak 43

9 Ariketak: ekosistema agenteekin 45

9.1 Ariketa ebatziak 45

9.2 Proposatutako ariketa gehigarriak 45

10 DevOps: egiaztapena eta berrikuspena . 47

10.1 DevOps ikuspegia: integrazio eta entrega jarraitua agenteekin 48

10.2 Hiru egiaztapen-maila: makina, scrumban-agentea, gizakia48

10.3 *Pull request* bat irakurtzen 48

10.4 Agenteak idatzitako testak epaitzen 48

11 Ohiko arazoak eta errezetak 49

11.1 Agenteak «eginda» dio, baina testek huts egiten dute 50

11.2 Agenteak testak aldatu ditu pasa zitezen 50

11.3 Bi agentek fitxategi bera aldatu dute 50

11.4 Agenteak zereginaren mugak gainditu ditu 50

11.5 Zeregina lausoegia zen eta agenteak asmatu egin du 50

11.6 Berrikuspenen ilara pilatu da eta fluxua gelditu da 50

11.7 Agenteak testuingurua galdu du zeregin luze batean 50

11.8 Kostua espero baino askoz handiagoa izan da 50

11.9 Agenteak sekretu bat edo baimen gehiegi erabili ditu 50

12 Eranskina: txantiloiak eta konfigurazioa .. 51

12.1 Zeregin- eta *pull request*-txantiloiak 52

12.2 Proiektu-taularen zutabeak, mugak eta arauak 52

12.3 Lerro nagusiaren babes-arauak 52

12.4 Integrazio jarraituaren *workflow*-a 52

12.5 Agenteen testuinguru-fitxategia 52

12.6 Scrumban-agentearen jarraibideak 52

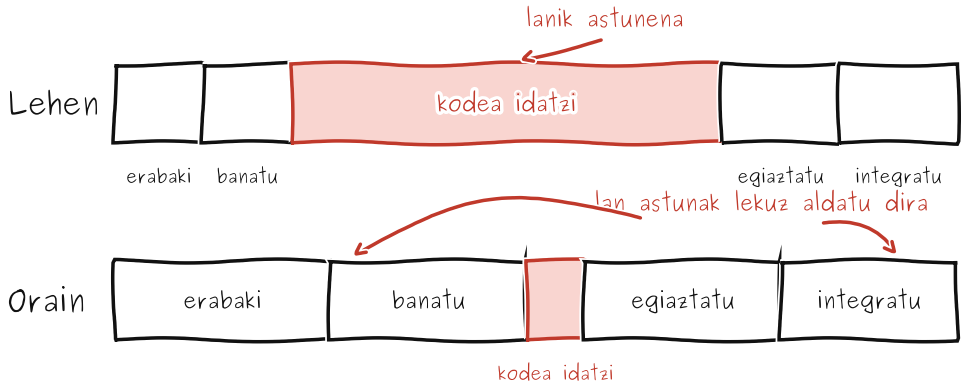
1. Hitzaurrea eta helburuak

Urte askotan, software-proiektu baten atal astunena kodea idaztea izan da. Orduak eta orduak teklatuaren aurrean, akatsen bila, funtzionalitate bat amaitu nahian. Hori bera ikusi izan dugu behin eta berriz Informatika Fakultateko eskoletan: azken egunetako super-heroi kodeketak, presakako integrazioa talde-lanetan eta taldekideen lana balidatzeko edota onartzeko arazoak. Horregatik idatzi genuen *Git bertsioak kontrolatzeko sistemarako eskuliburua* (Lopez-Gazpio eta Soroa Etxabe 2024): lana ez galtzeko, iterazio inkrementaletan pausoka garatzeko eta taldean integratzeko tresna bat eskura jartzeko.

Egun software-ingeniaritza aldatu da, eta oso azkar, gainera. Kodetzeko agenteek –hizkuntza-eredu handietan oinarritutako programek, iturburu-kodea idazteko, exekutatzeko eta zuzentzeko gai direnek– minutu gutxitan egiten dute duela gutxi ikasle bati arratsalde osoa eskatzen zion lana. Ez dizut gezurrik esango: **kodea idaztea ez da jada lanik astunena**. Hala eta guztiz ere, proiektuek lehengo modura porrot egiten jarraitzen dute; batzuetan, akaso, azkarrago.

Har dezagun ohiko kasu bat, adibide gisa. Lau ikasleko talde batek *sprint* oso bat "amaitu" du arratsalde batean, agente bati esker. Aurkezpenaren egunean demoak huts egiten du. Inork ez daki azaltzen kodeak zer egiten duen. Testak pasatzen dira, bai, baina norbaitek –edo zerbaitek– testak aldatu ditu arazorik egon ez dadin. Proiektu-taulan dena dago bukatuta egongo balitz modura anotatuta. Kodea ez zen arazoa, arazoa antolakuntza izan da. Inork ez zituen egin beharrekoak zehaztu, inork ez zuen lana zeregin argitan banatu, eta inork ez zuen egindakoa egiaztatu.

Software-ingeniaritzaren atal kritikoak lekuz aldatu dira: kodea idaztetik, **zer idatzi erabakitzera**, lana **zereginetan banatzera**, egindakoa **egiaztatzera** eta guztia **integratzera**. Gaur egun arreta diseinura, koordinaziora eta egiaztapenera bideratu behar da software-ingeniaritza profesionalean (1.1 irudia). Agenteek kodea idazten dute, baina ez dute zure ordez erabakitzen, ez dute zure taldea antolatzen eta ez dira zure lanaren erantzule. Hori guztia gizakion esku dago, lehen baino gehiago.



1.1. irudia: Lan astunak lekuz aldatu dira: lehen kodea idaztea zen lanik astunena; orain erabakitzea, zereginetan banatzea, egiaztatzea eta integratzea dira.

Eskuliburu honek **lantalde hibridoetan** lan egiten irakatsi nahi dizu: gizakiz eta agentez osatutako taldeetan. Ikusiko duzunez, agenteek lantalde on batek beti behar izan duen gauza bera behar dute, baina zorrotzago: zeregin argiak, onarpen-irizpideak, arauak dituen biltegi bat, epaile gisa jokatzen duen integrazio jarraitua eta berrikuspena. Agenteak ez du anbiguotasunik barkatzen: zeregin lausoa ematen badiozu, kode okerra idatziko dizu, eta konfiantza osoz gainera.

Ez dizut gezurrik esango, ordea: eskuliburu honek ez zaitu agenteen aditu bihurtuko, eta ez dizu tresna jakin baten erabilera irakatsiko. Tresnak hilabete gutxitan aldatzen dira, eta liburu hau inprimatzerako batzuk zaharkituta egongo dira. Horregatik, tresna zehatzak eranskinean eta liburuaren biltegian aipatzen dira, eta liburuan **aldatzen ez diren kontzeptuak** landuko dira: nola zehaztu, nola antolatu, nola egiaztatu, etab Eta, Gitekin bezala, hau ere **erabiliz ikasten da**: bidean ariketa-sorta handia landu beharko duzu, eta ariketa horietako askotan zuk zeuk egingo duzu eskuz gero agenteari emango diozun lana. Ez dago bestelako misteriorik.

Bada hasieratik aipatu nahi dudan arrisku bat. Agenteei lana eskuordetzea erraza da; ulermena eskuordetzea, berriz, hondamendia. Kodea irakurtzen, testak idazten eta

arkitektura epaitzen jakin behar duzu, lehen bezainbeste edo gehiago, zuk egiaztatu beharko baituzu agenteak "eginda" dioen hori benetan eginda dagoen. Eskuliburu honek **ardura** irakatsi nahi du, ez delegazioa. Ez zaitut agenteak erabiltzera behartu nahi; erabiliko dituzu, nahi ala ez, zure lankideek erabiliko dituztelako.

1.1 Nondik gatozen, non gauden

Eskuliburu honen aurrekari zuzena *Git bertsioak kontrolatzeko sistemarako eskuliburua* da, 2024an UEUk eta UPV/EHUk argitaratua. Liburu hark hutsune bat bete nahi izan zuen: ez zegoen euskarazko ikasmaterialik bertsioak kontrolatzeko sistemen inguruan, eta Informatika Fakultatean irakasgai askotan erabiltzen da Git. Git ardatz hartuta, liburuak garapen-metodologiekin, talde-lanarekin eta kode-kalitatearekin lotzen zuen tresna, eta, hain zuzen, lotura horietatik sortu da esku artean duzun liburu hau: nola antolatu software-proiektu handi bat, nola banatu, nola integratu, etab.

Software-ingeniaritza erabat aldatu da azken urteetan. Kodetzeko agenteak industrian eta ikasgeletan hedatu dira azkar, eta, hitzaurre honen hasieran esan dudan bezala, lan astunena diseinura, koordinaziora eta egiaztapenera mugitu da. Horrek esan nahi du software-ingeniaritzaren gaitasun klasikoak –erabiltzaile-historioak, UML, taldeen rolak, metodologia arinak, integrazio jarraitua– lehen baino beharrezkoagoak direla, baina lantalde berri baten kontestuan kokatzen dira: gizakiz eta agentez osatutako lantalde hibridoan, hain zuzen ere.

Gaur egun ez dago ikasmaterialik lantalde hibrido horiek antolatzen irakasten duenik, ez euskaraz eta ia ez beste hizkuntzetan ere. Atzigarri dauden baliabideak bi motatakoak dira: alde batetik, tresna zehatzen dokumentazioa, hilabete gutxitan zaharkitzen dena, eta, bestetik, garapen hibridorako lehen proposamen akademikoak, hala nola *Agentsway* (Bandara et. al. 2025), agenteak taldekide oso gisa hartzen dituena, fase bakoitzerako agente espezializatuekin. Liburu honen proposamena apalagoa da eta irakasteko egokiagoa: ikasleek dagoeneko ezagutzen dituzten bi metodo hartu –*Scrum* eta *Kanban*– eta lantalde hibridorako egokitu. **Scrumban agentikoa** deritzogu lan egiteko era horri (7. kapitulua): *Scrum*ek gizakien erritmoa antolatzen du –rolak, sprintak, berrikuspena– eta *Kanban*ek agenteen exekuzioa –zereginen fluxua, lanean dauden atazen mugak, arau esplizituak–.

Egiturari dagokionez, liburu honek aurreko eskuliburuaren bidea jarraitzen du: euskaraz, praktikoa, ariketa ebatziekin eta eskuz egindako irudiekin, errazenetik zailenera. Eta aurrekoaren osagarria da, ez ordezkoa.

1.2 Zer ikasiko dugu

Jorratuko ditugun gaien pintzelada bat ematearren: hasieran **taldeez eta rolez** mintzatuko gara (2. kapitulua), Belbinen rolak eta *Scrum*eko rolak gogoratuz, eta agenteak taldean non kokatzen diren erabakiz. Ondoren, **erabiltzaile-historioetatik zereginetara** pasatuko gara (3. kapitulua): nola idatzi agente batek –edo taldekide batek– zalantzarik gabe egikari dezakeen zeregin bat, onarpen-irizpideekin eta egindakoaren definizioarekin. Jarraian, **biltegia** lan-eremu partekatu gisa antolatuko dugu (4. kapitulua): zereginaren araberako adarrak, *pull request*-ak, babes-arauak eta integrazio jarraitua epaile gisa izanik.

Teoriarik gabeko kapitulu batean **eskuak zikinduko ditugu** (5. kapitulua): proiektu-taula bat sortuko dugu GitHuben, *Kanban* eta *Scrum* ikuspegiak gehituko dizkiogu, *issue*-ak idatzi eta zutabeetan zehar mugituko ditugu, eta *sprint* oso bat simulatuko dugu rol-jokoan. Hau da, agenteek gero modu automatiko batean kontsumituko duten informazio guztia nola sortzen den eskuz aztertuko dugu.

Behin oinarriak sendo ditugula **kodetzeko agenteen** inguruan mintzatuko gara (6. kapitulua): zer diren, nola ematen zaien testuingurua, zer baimen behar dituzten, zenbat kostatzen diren eta nola egiten duten huts. Horren gainean eraikiko dugu liburuaren muina, **Scrumban agentikoa** (7. kapitulua), metodoaren mekanika osoarekin: nondik datorren izena, zergatik nahasten diren *Scrum* eta *Kanban*, zereginen fluxua, lanean dauden zereginen muga, arau esplizituak eta eskalatze-protokoloa. Gainbegirale-agente batek, scrumban-agenteak, zainduko du fluxua, eta gizakiak erabakiko ditu garapenaren alderdi kritikoak.

Deskribatu berri dugun fluxua bi ariketa-kapitulutan jarriko dugu praktikan: lehenbizi, **kaixo, Scrumban agentikoa!** erako ariketa txiki batekin (8. kapitulua) –zeregin bat, agente bat, *pull request* bat–, eta gero, **ekosistemaren proiektuarekin** (9. kapitulua), proiektu konplexu bat agenteen bitartez aurre eginda. Amaieran, **DevOps** ikuspegitik egiaztapenaz eta berrikuspenaz arituko gara (10. kapitulua) –gizakiak zer egiaztatu behar duen, eta nola–, eta **ohiko arazoen eta errezeten** kapitulu batekin itxiko dugu eskuliburua (11. kapitulua). Txantiloiak eta konfigurazio-fitxategiak eranskinean bildu ditugu (12. kapitulua).

1.3 Helburuak

Kodetzeko agenteek minutu gutxitan egiten dute duela gutxi garatzaile bati arratsalde osoa lanean jo eta ke lanean egotea eskatuko liokeen kodeketa-lana. Hala ere, software-proiektuek lehengo modu berean egiten dute porrot. Lan astunak birmoldatu egin dira: kodea idaztetik zer eraiki erabakitzera, lana zereginetan banatzera, egindakoa egiaztatzera eta integratzera. Hau da, software-ingeniaritzara. Eskuliburu honek lantalde hibridoetan (gizakiz eta agentez osatutakoetan) lan egiten irakasteko helburua du: taldeak eta rolak antolatzen, agente batek zalantzarik gabe egikari dezakeen zeregina idazten, biltegia arauekin eta ateekin prestatzen, eta agenteen lana gainbegiratzen eta egiaztatzen. Muina Scrumban agentikoa da, *Scrum* eta *Kanban* lantalde hibridorako egokitzen dituen lan egiteko modua: *Scrum* gizakien erritmorako, *Kanban* agenteen exekuziorako. *Git Bertsioak Kontrolatzeko Sistemarako eskuliburua*ren (2024) jarraipena da, egitura eta estilo berarekin: euskaraz, praktikoa, ariketa ebatziekin eta eskuz egindako irudiekin. Software proiektu handiak bideratzeko kemena duen edonori zuzenduta dago, eta tresnak zaharkitu arren aldatzen ez dena irakatsi nahi du: nola zehaztu, nola antolatu, nola egiaztatu.

Zehazki, eskuliburu hau amaitzean honakoak egiteko gai izatea gustatuko litzaiguke:

- agente batek edo taldekide batek anbiguotasunik gabe egikari dezakeen zeregin bat idazteko, onarpen-irizpide eta guzti;
- biltegi bat antolatzeko, zereginaren araberako adarrekin, berrikuspenarekin eta integrazio jarraituarekin;
- proiektu-taula bat kudeatzeko, *Kanban* nahiz *Scrum* ikuspegian;
- agenteen lana gainbegiratzeko Scrumban agentikoaren fluxuan: zereginak esleitu, egoera-txostenak irakurri, blokeoak eskalatu eta lana itxi;
- agente batek egindako lana egiaztatzeko: *diff*-ak irakurri, testak epaitu, arkitektura zaindu;
- agenteen huts egiteko modu ohikoenak antzemateko eta konpontzeko.

Eskuliburua edozein programazio-hizkuntzatan kodetzen dakien eta Git erabili duen edonorentzat da. Giten oinarriak falta bazaizkizu, hasi aurreko eskuliburutik; hemen ez ditugu errepikatuko. Gogoratu eskuliburu bat ez dela zertan hasieratik bukaerara irakurri behar: taldea antolatzea bada zure larria, hasi 2. kapitulutik; agenteekin probatzeko gogoz bazaude, 5. kapituluko ariketak dira abiapuntu ona, 7. eta 8. kapituluetara salto egin aurretik. Txantiloiak, konfigurazio-fitxategiak eta ariketen biltegiak liburuaren biltegian daude, eguneratuta.

Prest zara bideari ekiteko?

1.4 Eskerrak

Eskerrak idazteko, liburua amaitzean.

2. Taldeak eta rolak

Laburpena (behin-behinekoa): Kapitulu honetan lantaldea osatuko dugu, agenteak sartu aurretik. Belbinen talde-rolak gogoratuko ditugu (Belbin 2010) —koordinatzailea, moldatzailea, landarea, monitorea, taldekidea, inplementatzailea, amaitzailea, baliabideen ikertzailea eta espezialista— eta ikasleek beren indarguneak eta ahulguneak identifikatzeko erabiliko ditugu. Ondoren, *Scrum*eko rolekin lotuko ditugu: produktu-jabea, *Scrum* zuzendaria eta garatzaileak. Azkenik, galdera berria egingo dugu: agenteak taldean sartzean, zer rol betetzen dute eta zer rol ez? Kapituluaren tesia da agenteek inplementatzailearen eta amaitzailearen lana hartzen dutela neurri handi batean, eta gizakiei koordinatzailearen, monitorearen eta produktu-jabearen lana geratzen zaiela, lehen baino garrantzitsuago. Ariketa batekin itxiko dugu: taldearen rol-mapa, agenteak barne.

2.1 Belbinen talde-rolak

2.2 *Scrum*eko rolak: produktu-jabea, *Scrum* zuzendaria, garatzaileak

2.3 Agenteak taldean: zer rol bai, zer rol ez

2.4 Ariketa: taldearen rol-mapa, agenteak barne

3. Erabiltzaile-historioak eta zereginak

Aurreko eskuliburuan esan genuen *agile* metodologietako erabiltzaile-historioak Giten adar gisa garatzen direla, eta erabiltzaile-historio onek *INVEST* izan behar dutela (Cohn 2004): independenteak, negoziagarriak, baliotsuak, estimagarriak, neurgarriak eta probagarriak. Orduan nahikoa zen hori, kodea idaztea geure esku zegoelako eta hutsuneak buruan betetzen genituelako kodetzen ari ginen bitartean. Orain ez da nahikoa. Zeregin bat agente bati –edo ezagutzen ez zaituen taldekide bati– ematen diozunean, idatzi edo formalizatu ez duzun hori guztia asmatu egingo du; eta asmatu, ziur asko, zuk nahi ez zenuen moduan. Kapitulu honetan bide osoa egingo dugu, erabiltzaile-historiotik zereginera: erabiltzaileak zer nahi duen eta zergatik; hortik, zer egin behar den zehazki; eta zer bete behar den eginda dagoela esateko.

Bide horretan bada tarteko pauso bat, askotan saltatzen dena: **diseinua**. Historio batek esaten du zer lortu nahi den; zereginak esaten du zer egin; baina bien artean norbaitek erabaki behar du **nola**: zer modulu edo klase egongo diren, nola hitz egingo duten elkarrekin, zer egoeratatik igaroko den sistema. Erabaki horiek kodean bakarrik hartzen baditugu, lehen konpilatzen den inplementazioak erabakiko du diseinua, eta ez taldeak. Agenteekin, gainera, lehen konpilatzen den inplementazioa minutu batean iritsiko da. Horregatik behar dugu erabakiak kodea baino lehen hartzeko mekanismo arautua, eta kodetik kanpo idazteko modu zehatza, eta, hain zuzen ere horretarako sortu zen UML.

UML (*Unified Modeling Language*) softwarea modelatzeko hizkuntza grafiko

estandarra da. 1990eko hamarkadan sortu zen, Grady Boochen, James Rumbaughen eta Ivar Jacobsonen modelatze-metodoak bateratuz, eta 1997tik *Object Management Group* (OMG) erakundeak estandarizatzen du (Object Management Group 2017). Diagrama mota askok osatzen dute UML, bi familiatan banatuta: **egiturazkoak**, sistema zer osagaiz osatuta dagoen adierazten dutenak –klase-diagrama, osagaien diagrama, hedapen-diagrama–, eta **portaerazkoak**, sistemak nola jokatzen duen adierazten dutenak –erabilera-kasuen diagrama, sekuentzia-diagrama, jarduera-diagrama, egoera-makinaren diagrama–. Ez dizut gezurrik esango: UML osoa ikastea liburu oso baten lana da, eta badira horretarako material bikainak (esaterako, Fowler 2004). Hemen lau diagrama baino ez ditugu erabiliko, eta bakoitza arrazoi zehatz batekin.

Zertarako behar dugu UML garapena kontrolatzeko? Hiru arrazoirengatik. Lehenik, diagrama bat **kodea baino lehen hartutako erabakia** da: klase-diagrama batek interfazeak finkatzen ditu agenteak lehen lerroa idatzi aurretik, eta sekuentzia-diagrama batek zehazten du nork deitzen dion nori. Bigarrenik, diagrama bat **hizkuntza partekatua** da: taldekide guztiek —eta agenteek— gauza bera ulertzen dute marrazki beretik, eta eztabaidak marrazkiaren gainean egiten dira, ez kodearen gainean, askoz merkeagoa eta zehatzagoa dena. Hirugarrenik, diagrama bat **egiaztagarria** da: zeregin batean klase-diagrama bat badago, berrikusleak egiazta dezake inplementazioak diagrama errespetatzen duen, eta agenteari eska diezaiokegu errespetatzeko. Beste hitz batzuetan: diagramak zereginaren **kontratuaren** parte dira, onarpen-irizpideekin batera.

Kontuz, ordea: ez baitugu eredu osorik egingo, ezta diagramak kodearekin batera eguneratuta mantentzeko agintzarik ere, hori ez baita errealitatean askotan betetzen. Agile metodologiaren kasuan, **UML arina** egingo dugu: diagrama bat marraztuko dugu soilik agentearen edo taldekidearen lana mugatzen duenean, eta zeregina ixtean diagrama hori bukatutzat jo.

Hona hemen kapituluaren ibilbidea: lehenbizi erabiltzaile-historioak gogoratuko ditugu eta *INVEST* irizpideak berrikusiko ditugu agenteen ikuspegitik; ondoren, onarpen-irizpideak eta egindakoaren definizioa idazten ikasiko dugu; gero, zeregina kontratu gisa aztertuko dugu, *issue* baten anatomia osoarekin; eta, amaitzeko, UML arina txertatuko dugu zereginetan, lau diagramekin: klase-diagrama, sekuentzia-diagrama, erabilera-kasuen diagrama eta egoera-makinaren diagrama.

3.1 Erabiltzaile-historioak eta *INVEST*, agenteen ikuspegitik

3.2 Onarpen-irizpideak eta egindakoaren definizioa

3.3 Zeregina kontratu gisa: *issue* baten anatomia

3.4 UML arina: klase-, sekuentzia-, erabilera-kasu- eta egoera-makinaren diagramak

4. Biltegia, lan-eremu partekatua

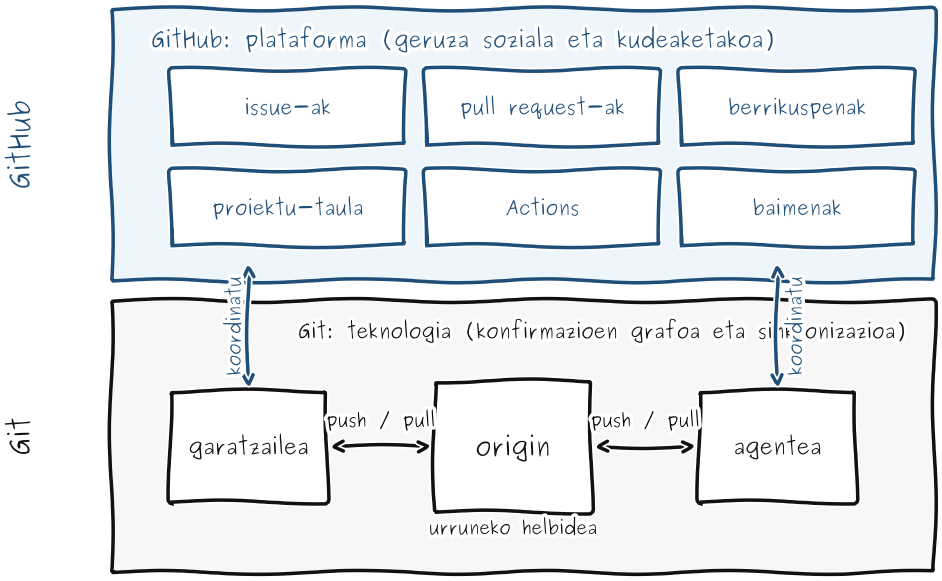
Git bertsioak kontrolatzeko sistema **banatua** da, hau da: nodo bakoitzak —garatzaile bakoitzaren ordenagailuak— historiaren kopia osoa du, eta besteen beharrik gabe lan egin dezake. Taldean lan egiteko, ordea, nodoek noizbait sinkronizatu behar dute, eta horretarako guztiek **urruneko helbide** bera erabiltzen dute, Giten *origin* izena hartu ohi duena. Hodeian dagoen biltegi hori beste nodo bat da sistemaren barruan, berezitasun batekin: taldekide guztiek ezagutzen dute, eta beste guztiak harekin sinkronizatzen dira, konfirmazioak eta adarrak igoz eta jaitsiz. Helbide hori `git clone` aginduarekin jasotzen dugu kopia bat egitean, edo `git remote add origin` aginduarekin lotzen diogu lokalean hasieratutako biltegi bati; hortik aurrera, `git push`, `git fetch` eta `git pull` aginduek egiten dute gainerakoa. Ez ditut hemen agindu horiek errepikatuko; zalantzarik baduzu, jo aurreko eskuliburuaren adarkatzeari buruzko kapitulura.

Bada, ordea, aurreko eskuliburuan nahita lausotuta utzi genuen bereizketa bat, eta orain argitu beharra daukagu: **Git eta GitHub ez dira gauza bera**.

Git teknologia da. Zure ordenagailuan exekutatzen den programa bat da, konfirmazioen grafo bat gordetzen duena eta grafo horiek nodoen artean sinkronizatzeko protokolo bat eskaintzen duena. Gitek ez daki nor zaren, konfirmazio bakoitzean idatzitako izenetik eta helbide elektronikotik harago; ez daki zer baimen dituzun; ez daki zer den *issue* bat, berrikuspen bat edo proiektu-taula bat. *Origin* helbide bat besterik ez da, eta edozein tokitara apunta dezake: USB memoria bateko karpeta batera, unibertsitateko zerbitzari batera edo hodeiko plataforma batera. Gitentzat

guztiak berdinak dira (4.1 irudia).

GitHub plataforma da: urruneko biltegiak ostatatzen dituen zerbitzu bat, Giten gainean **geruza sozial eta kudeaketakoa** eraikitzen duena. Geruza horretan daude nortasuna eta baimenak —nor den taldekide, nork idatz dezakeen lerro nagusian—, *issue*-ak, *pull request*-ak, berrikuspenak, proiektu-taulak, adarren babes-arauak, integrazio jarraitua (*GitHub Actions*) eta bertsioen argitalpena. Hau da, liburu honetan konfirmazioen grafoaren gainetik ikusiko dugun ia guztia GitHuben geruzakoa da, ez Gitena. GitHub ez da bakarra: GitLab, Gitea edo Bitbucket plataformek antzeko geruza eskaintzen dute, izen eta xehetasun desberdinekin. GitHub aukeratu dugu nagusia delako eta hezkuntzarako doakoa delako, baina kontzeptu guztiak eramangarriak dira.



4.1. irudia: Git teknologia da: konfirmazioen grafoa eta nodoen arteko sinkronizazioa. GitHub Giten gainean eraikitako geruza soziala eta kudeaketakoa da, eta gizakiek zein agenteek bi geruzekin hitz egiten dute.

Bereizketa hori ez da tribiala; lantalde hibrido batean funtsezkoa da. Agenteek bi geruzekin hitz egiten dute, eta bakoitzarekin modu desberdinean egiten dute, gainera: Gitekin, kodea idazteko eta sinkronizatzeko —adarrak, konfirmazioak, *push*—; plataformarekin, koordinatzeko —zereginak hartu, egoera jakinarazi, *pull request*-ak ireki, berrikuspenak jaso—. Gitek ez dio inori ezer galdetzen; plataformak bai: baimenak, arauak eta ateak plataforman bizi dira, eta horiek dira agente bati lerro nagusia hausteko aukera kentzen dioten mekanismoak. Horregatik diogu biltegia

lan-eremu partekatua dela: ez kodea gordetzen den tokia bakarrik, baizik eta gizakiek eta agenteek elkarrekin lan egiteko arauak idatzita dauden tokia.

Kapitulu honetan lau gauza antolatuko ditugu lan-eremu horretan: zereginaren araberako **adarrak**, agente bakoitzak bere adarrean lan egin dezan; ***pull request*-a** berrikuspen-unitate gisa, lana lerro nagusira iristen den ate bakarra; lerro nagusiaren **babes-arauak**, inork —ez gizakiak, ez agenteak— zuzenean idatz ez dezan; eta **integrazio jarraitua** epaile gisa, *pull request* bakoitzean testak exekutatzen dituena eta emaitza gorriz edo berdez erantzuten duena.

4.1 Urruneko helbidea: Git teknologia, GitHub plataforma

4.2 Zereginaren araberako adarrak

4.3 *Pull request*-a berrikuspen-unitate gisa

4.4 Lerro nagusiaren babes-arauak

4.5 Integrazio jarraitua epaile gisa

5. Ariketak: zereginak eta taulak GitHuben

Kapitulu honetan ez dugu teoria berririk ikasiko. Aurreko lau kapituluetan ikusitakoa eskuekin finkatuko dugu, agenteei zereginik eman aurretik: proiektu-taula bat sortuko dugu GitHuben, Kanban eta Scrum ikuspegiak gehituko dizkiogu, *issue*-ak idatziko ditugu eta zutabeetan zehar mugituko ditugu, eta adarrak eta *pull request*-ak *issue*-ei lotuko dizkiegu. Agenteek gero kontsumituko duten guztia gizakiok egingo dugu lehenbizi, pausoz pauso, zer gertatzen den ulertzeko. Ariketa hauetan ere ekosistemaren proiektua eta Nami, Ane, Mikel eta Gotzonen taldea izango ditugu lagun.

5.1 Ariketa ebatziak

5.1.1 1. ariketa - Biltegia eta proiektu-taula sortzen

Sortu taldearen biltegia GitHuben, sortu *GitHub Projects* taula berri bat eta lotu biltegiari. Gehitu taldekide guztiak, bakoitza bere rolarekin. Zer alde dago erabiltzaile baten proiektuaren eta erakunde baten proiektuaren artean?

Biltegia sortzea da errazena, eta aurreko eskuliburuan ikasitakoa baino ez da: GitHuben *New repository* aukeratu, izena eman —`ekosistema`, esaterako—, *README* fitxategi batekin eta lizentzia batekin hasieratu, eta urruneko helbidea kopiatu. Terminaletik gauza bera egin daiteke `gh` tresnarekin, agindu bakarrean, eta kopia

lokala ere sortuta:

```
gh repo create ekosistema --public --add-readme \

                --license mit --clone
```

Taldekideak gehitzeko, biltegiaren *Settings* atalean *Collaborators* aukeratu eta bakoitza bere erabiltzaile-izenarekin gonbidatu. Idazteko baimena (*write*) nahikoa da garatzaileentzat; administratzailearena taldeko batek bakarrik behar du, eta hobe da gutxi izatea, 4. kapituluan ikusi dugun bezala, lerro nagusiaren babes-arauak administratzaileek bakarrik alda baititzakete.

Proiektu-taula sortzeko, erabiltzailearen edo erakundearen *Projects* fitxan *New project* aukeratu, *Board* txantiloia hautatu eta izena eman. Kontuz: proiektua sortu berritan **ez dago biltegiari lotuta**. Biltegiaren *Projects* fitxan *Link a project* aukeratu behar da, edo terminaletik:

```
gh project create --owner @me --title "Ekosistema"

gh project link 1 --owner @me --repo erabiltzailea/ekosistema
```

Lotura horri esker, biltegiko *issue*-ak taulara gehi daitezke eta taulako automatizazioek biltegiko gertaerei erantzun diezaiekete; lotu gabe, taula zerrenda soil bat da, ezer gertatzen ez den tokia. Taldekide bakoitzak taula ikusten eta editatzen duela egiaztatu behar da amaitu aurretik; proiektuek beren baimenak dituzte, biltegiarenetatik bereizita daudenak.

Erabiltzaile batek sortutako proiektua kontu horrena da —hark bakarrik administra dezake, eta kontua desagertzen bada, proiektua ere bai—, eta biltegi bakarrarekin edo gutxirekin lotu ohi da. Erakunde batek sortutako proiektua erakundearena da: taldeei (*teams*) baimenak eman dakizkieke, erakundeko biltegi guztietako *issue*-ak bil ditzake, eta pertsonak joan arren proiektuak bere horretan jarraitzen du. Ikasgela baterako, beraz, erakunde bat sortzea merezi du. Interfazearen izenak aldatu egiten dira noizean behin; kontzeptuak —biltegia, lotutako proiektua, baimenak— ez.

5.1.2 2. ariketa - Kanban ikuspegia eta lanean dauden zereginen muga

Sortu Kanban ikuspegi bat *Status* eremuan oinarrituta, hiru zutaberekin: Egiteko, Egiten eta Eginda. Jarri muga bat Egiten zutabeari. Zergatik mugatu behar da aldi

berean lanean dauden zereginen kopurua?

Ebazpena garatzeko.

5.1.3 3. ariketa - Scrum ikuspegia: iterazioak, lehentasunak eta estimazioak

Gehitu taulari *Iteration* eremu bat sprintak definitzeko, eta *Priority* eta *Estimate* eremu pertsonalizatuak. Sortu taula-ikuspegi bat iterazioka taldekatuta eta *Roadmap* ikuspegi bat. Noiz da erabilgarriagoa Kanban ikuspegia, eta noiz Scrum ikuspegia?

Ebazpena garatzeko.

5.1.4 4. ariketa - *Issue*-ak idazten

Idatzi ekosistemaren *product backlog*-eko lehen zereginak *issue* gisa, txantiloi bati jarraituz: testuingurua, onarpen-irizpideak eta egindakoaren definizioa. Gehitu etiketak (*bug*, *feature*, *docs*), arduraduna eta mugarria. Betetzen ditu zure *issue* bakoitzak *INVEST* irizpideak?

Ebazpena garatzeko.

5.1.5 5. ariketa - *Issue*-ak zutabeetan zehar mugitzen

Mugitu *issue*-ak eskuz zutabeetan zehar, lana aurrera doan heinean. Egoera-aldaketa bakoitzean idatzi iruzkin labur bat: zer egin den, zer falta den eta zerk blokeatzen duen. Iruzkin horiek dira gero agenteek emango dituzten egoera-txostenak.

Ebazpena garatzeko.

5.1.6 6. ariketa - Taularen automatizazioak

Konfiguratu taularen *workflow* integratuak: *issue* berria Egiteko zutabera, *issue* itxia edo *pull request* bateratua Eginda zutabera. Egiaztatu aurreko ariketan eskuz egiten

zenuena orain automatikoki gertatzen dela. Zer gelditzen da oraindik eskuz egiteko?

Ebazpena garatzeko.

5.1.7 7. ariketa - *Issue* batetik adarrera eta *pull request*-era

Sortu adar bat *issue* batetik, egin konfirmazioak *issue*-a aipatuz (`#N`), ireki *pull request* bat `Closes #N` esaldiarekin, eskatu berrikuspena eta bateratu. Egiaztatu *issue*-a itxi dela eta txartela Eginda zutabean dagoela. Marraztu taldearen Giten historia-grafoa bateratu aurretik eta ondoren.

Ebazpena garatzeko.

5.1.8 8. ariketa - Sprint baten simulazioa rol-jokoan

Simulatu sprint oso bat rol-jokoan: Ane produktu-jabea, Gotzon gainbegiralea, Nami eta Mikel garatzaileak. Planifikatu *sprint backlog*-a, egin eguneroko bilera taularen aurrean, jarraitu *burndown* grafikoa *Insights* atalean, itxi sprinta eta egin atzera-begirakoa. Gizakiek egingo dute hemen gero agenteek egingo dutena.

Ebazpena garatzeko.

5.1.9 9. ariketa - Blokeoak eta eskalatzea

Sortu *blocked* eta *needs-human* etiketak, eta erabili aurreko ariketako sprintean. Marraztu paperean *issue* baten egoera-makina: egoerak, trantsizioak eta nork eragiten duen bakoitza. Egoera-makina hori formalizatuko dugu 7. kapituluan.

Ebazpena garatzeko.

5.1.10 10. ariketa - Gauza bera terminaletik

Errepikatu 4., 5. eta 7. ariketak terminaletik, `gh` komando-lerroko tresna erabiliz: `gh issue create`, `gh issue edit`, `gh project item-add` eta `gh pr create`.

Hau da agenteek erabiliko duten interfazea; hemendik aurrera, taulan ikusten duzun guztia agindu bihur daiteke.

Ebazpena garatzeko.

5.2 Proposatutako ariketa gehigarriak

Hona hemen zereginen taularekin trebatzen jarraitzeko proposatzen diren ariketa gehigarriak:

1. Idatzi *issue*-txantiloi bat eta *pull request*-txantiloi bat biltegiaren `.github` karpetan. Zer onura du txantiloiak agenteek *issue*-ak kontsumituko dituztenean?

2. Idatzi zutabe bakoitzaren arauak (noiz sar daiteke txartel bat zutabe batean, noiz atera) dokumentu labur batean biltegian. Arau horiek izango dira gero gainbegiralearen irizpideak.
3. Gehitu *needs-human* zutabe bat eta jarri 1eko muga. Zer gertatzen da fluxuarekin muga horrekin? Eta mugarik gabe?
4. Neurtu *issue* bakoitzaren ziklo-denbora, Egiten zutabera sartzen denetik Eginda zutabera iristen den arte. Zein da zereginik motelena, eta zergatik?
5. Taula bera Kanban ikuspegian eta Scrum ikuspegian kudeatu bi aste segidan. Zein da egokiagoa taldearen proiektuarentzat? Zein izango litzateke egokiagoa agenteentzat?
6. Sortu *issue* bat beste *issue* batzuen menpekoa dena, eta adierazi menpekotasuna. Nola kudeatzen du taulak? Nola jakin dezake agente batek zeregin bati ekin diezaiokeen ala ez?

6. Kodetzeko agenteak

Laburpena (behin-behinekoa): Kapitulu honetan kodetzeko agenteak aurkeztuko ditugu, tresna zehatzetan sartu gabe. Lehenik, zer diren azalduko dugu: hizkuntza-eredu handi bat, tresna batzuk —fitxategiak irakurri eta idatzi, aginduak exekutatu, biltegiarekin eta plataformarekin hitz egin— eta begizta bat, helburua lortu arte planifikatu, egin eta egiaztatu egiten duena. Ondoren, agente batek lan egiteko behar duena: testuingurua (proiektuaren arau-fitxategia, zeregina, kodea), baimenak (zer egin dezakeen eta zer ez), isolamendua (non exekutatzen den) eta kostua (tokenak, denbora, dirua). Gero, nola hitz egiten dioten biltegiari eta proiektu-taulari, eta zergatik den hori 4. eta 5. kapituluetan egindako lanaren jarraipen naturala. Azkenik, huts egiteko moduak: frogarik gabeko "eginda"-k, testak ahultzea, zereginaren mugak gainditzea, testuingurua galtzea eta kostua lehertzea. Tresna zehatzen konfigurazioa eranskinean eta biltegi lagunean dago, zahartzen denean eguneratu ahal izateko.

6.1 Zer da kodetzeko agente bat: eredua, tresnak eta begizta

6.2 Testuingurua, baimenak, isolamendua eta kostua

6.3 Agenteak biltegiarekin eta proiektu-taularekin hizketan

6.4 Huts egiteko moduak

7. Scrumban agentikoa

Aurreko kapituluetan taldea osatu dugu, zereginak idazten ikasi dugu, biltegia antolatu dugu, proiektu-taula eskuz kudeatu dugu eta, azkenik, kodetzeko agenteak ezagutu ditugu. Falta dena da hori guztia lotzen duen **lan egiteko modua** definitzea da: nork erabakitzen duen zer egin, nork egiten du, nork egiaztatzen du eta zer erritmotan. Kapitulu honetan liburuaren muina aurkeztuko dizut: Scrumban agentikoa.

Izena bera azalpen bat da. *Scrumban* bi metodo ezagunen nahasketa baino ez da, *Scrum* eta *Kanban*, eta *agentikoa* adjektiboak esan nahi duena da nahasketa hori gizakiz eta agentez osatutako lantalde baterako egokitu dugula. Ez da zerotik asmatutako metodologia bat. Nahiago izan dut ezagunak diren eta funtzionatzen dutela frogatu duten bi oinarri sendo hartzea, eta behar dugun tokian bakarrik moldatzea. Horregatik, metodoa ulertzeko, lehenbizi osagaiak ulertu behar dira.

7.1 Nondik dator terminoa?

Scrumban hitza Corey Ladasek zabaldu zuen 2008 inguruan, bere blogean argitaratutako saiakera-sorta batean, gero liburu batean bildu zuena (Ladas 2009). Ladasen asmoa ez zen metodo berri bat sortzea, baizik eta *Scrum* erabiltzen zuten taldeei *Kanban*en eta *lean* pentsamoldearen ideiak pixkanaka txertatzeko bide bat eskaintzea: lana ikusgai jarri, aldi berean lanean dagoen lan kopurua mugatu eta fluxua neurtu, *Scrum*en egitura bat-batean bota gabe. Denborarekin, izena bi metodoen edozein

nahasketa praktiko izendatzeko erabiltzen hasi zen: *Scrum*en rolak eta gertaerak mantentzen dituzten taldeak, baina lana *Kanban* taula batean eta fluxuan oinarrituta kudeatzen dutenak. Gaur egun ohikoa da aipatutakoa industrian ikustea, bereziki garapen-lanak eta mantentze-lanak nahasten dituzten taldeetan.

Agentikoa adjektiboa, berriz, askoz berriagoa da. Adimen artifizialaren esparruan, *agente* deitzen diogu helburu bat emanda bere kabuz jarduten duen sistemari: planifikatu, tresnak erabili, emaitzak aztertu eta berriro saiatu, pausoz pauso, gizakiak une bakoitzean gidatzea behar izan gabe. 6. kapituluan ikusi dugun bezala, kodetzeko agenteak horrelakoak dira: iturburu-kodea idazten, exekutatzen eta zuzentzen dute, eta biltegiarekin, testekin eta proiektu-taularekin elkarreragiten dute. Software-ingeniaritza agentikoa, beraz, agenteak lantaldearen parte diren software-ingeniaritza da; eta Scrumban agentikoa, lantalde horretarako proposatzen dudan lan egiteko modua.

Ez naiz bakarra bide horretan. Agenteak taldekide oso gisa hartzen dituzten metodologia-proposamenak agertzen hasi dira; esaterako, *Agentsway* (Bandara et. al. 2025), fase bakoitzerako agente espezializatuak definitzen dituena, gizakiaren gainbegiradapean. Gure hautua apalagoa da eta, nire ustez, irakasteko egokiagoa: ikasleek dagoeneko ezagutzen dituzten bi metodo hartu, eta agenteek aldatzen dutena bakarrik aldatu.

7.2 *Scrum*: inplementazioaren erritmoa

Scrum lan-esparru bat da —ez metodo oso bat—, produktu konplexuak garatzeko, 1990eko hamarkadan Ken Schwaberrek eta Jeff Sutherlandek proposatua eta *Scrum* gidan laburbildua dago (Schwaber eta Sutherland 2020). Haren oinarria **enpirismoa** da: ez dugu dena aurrez planifikatzen, baizik eta ziklo laburretan lan egin, egindakoa aztertu eta plana egokitzen dugu. Horretarako, *Scrum*ek hiru gauza definitzen ditu: rolak, gertaerak eta artefaktuak.

Rolak hiru dira. **Produktu-jabeak** produktuaren balioaren ardura du: zer eraiki eta zer ordenatan erabakitzen du, eta bezeroarekin edo irakaslearekin hitz egiten du. **Garatzaileek** produktua eraikitzen dute, eta *sprint* bakoitzean zer eta nola egin erabakitzen dute. ***Scrum* zuzendariak** esparrua ulertzen eta aplikatzen laguntzen du, eta taldeak dituen oztopoak ebazten laguntzen du. Aurreko eskuliburuan rol horiek banatzen ikusi genuen talde txiki bat; hemen, agenteak sartzean rolak nola aldatzen diren ikusiko dugu.

Gertaerak *sprint*aren inguruan antolatzen dira. *Sprint*a iraupen finkoko zikloa da, astebetetik hilabetera bitartekoa, eta haren barruan sartzen dira beste guztiak: **sprint-planifikazioa**, non taldeak sprintaren helburua eta egingo duen lana aukeratzen dituen; **eguneroko *Scrum*a**, garatzaileen koordinazio-bilera laburra; **sprint-berrikuspena**, non egindakoa interesdunei erakusten zaien eta haien iritzia jasotzen den; eta **atzera-begirakoa**, non taldeak bere lan egiteko modua aztertzen eta hobetzen duen. Gertaera guztiek iraupen mugatua dute, *timebox* deritzona.

Artefaktuak ere hiru dira: ***product backlog*a**, produktuak behar duen guztiaren zerrenda ordenatua; ***sprint backlog*a**, uneko sprinterako aukeratutako lanaren zerrenda, planarekin batera; eta **gehikuntza**, sprintaren amaieran erabilgarri dagoen produktuaren bertsioa. Artefaktu bakoitzak konpromiso bat du lotuta, eta horietako bat ezinbestekoa izango zaigu: **egindakoaren definizioa**, hau da, lan bat benetan eginda dagoela esateko bete behar diren baldintzen zerrenda.

Zertarako balio du horrek guztiak? *Scrum*ek bi gauza bermatzen ditu, batez ere. Batetik, **erritmoa**: sprint bakoitzak hasiera eta amaiera garbiak ditu, eta amaieran zerbait erakusgarri egon behar da. Bestetik, **babesa**: sprintaren barruan taldea ez da etengabe aldatzen diren eskakizunen mende, eta aldaketak hurrengo planifikaziorako gordetzen dira. Bi ezaugarri horiek proiektu baten **inplementazioa antolatzeko** dira: taldeak zer eraiki eta noiz erabakitzeko, eta interesdunekin atzeraelikadura-ziklo erregularra izateko. Hori da, hain zuzen, Scrumban agentikoan *Scrum*i eskatuko dioguna.

7.3 *Kanban*: exekuzioaren fluxua

Kanban hitza japonierazkoa da, eta "seinale-txartela" esan nahi du. Toyotaren ekoizpen-sisteman sortu zen, XX. mendearen erdialdean, Taiichi Ohnoren eskutik (Ohno 1988): fabrikako postu bakoitzak txartel baten bidez eskatzen zion aurreko postuari material gehiago, eta soilik behar zuenean. Horrela, ekoizpena ez zen bultzatzen (*push*), tiratzen baizik (*pull*): inork ez zuen ezer ekoizten hurrengo postuak eskatu arte, eta inon ez zen pilatzen erdi egindako lanik. Software-garapenera David J. Andersonek ekarri zuen ideia hori, 2000ko hamarkadaren amaieran (Anderson 2010), eta ordutik *Kanban* deitzen diogu lan intelektuala fluxu gisa kudeatzeko metodoari.

*Kanban*ek ez du rolik, ez sprintik, ez gertaera finkorik agintzen. Haren muina **sei praktika** dira:

1. **Lana ikusgai jarri.** Zeregin bakoitza txartel bat da taula batean, eta taulako zutabeek lanaren egoerak adierazten dituzte: egiteko, egiten, berrikusteko, eginda...
2. **Aldi berean lanean dagoen lana mugatu.** Zutabe bakoitzak muga bat du (*WIP limit*, ingelesez), eta muga betea dagoenean ezin da txartel berririk sartu bat atera arte.
3. **Fluxua kudeatu.** Helburua ez da jendea lanpetuta edukitzea, zereginak hasieratik amaierara ahalik eta azkarren eta uniformeen igarotzea baizik.
4. **Arauak esplizitu egin.** Zutabe bakoitzak arau idatziak ditu: noiz sar daiteke txartel bat, noiz atera, eta nork erabakitzen duen.
5. **Atzeraelikadura-begiztak ezarri.** Taula aztertzeko eta fluxua egokitzeko bilera erregularrak, taldeak behar duen maiztasunarekin.
6. **Elkarlanean hobetu.** Aldaketak txikiak eta ebolutiboak dira, taldeak adostuak eta neurketetan oinarrituak.

Eta neurketak ditu *Kanban*ek beste ezerk baino gehiago. Bi dira nagusiak: **ziklo-denbora**, zeregin batek lanean hasi zenetik amaitu arte behar izan duen denbora; eta **emaria**, denbora-tarte batean amaitutako zereginen kopurua. Aldi berean lanean dagoen lana mugatzeak ziklo-denbora laburtzen du, eta hori da ideia osoaren gakoa: zenbat eta zeregin gehiago izan aldi berean irekita, orduan eta gehiago itxaroten du bakoitzak, eta orduan eta gehiago aldatzen gara batetik bestera ezer bukatu gabe.

Kanban **exekuziorako** metodoa da, beraz: ez du eraiki behar den produktuaren inguruan ezer esaten, ez eta erabakiak hartu behar dituenaren inguruan, atazaren bizi zikloaren inguruan bakarrik agintzen du, hau da: garbi uzten duen gauza bakarra da lanak nola igaro behar duen sisteman zehar. Horrexegatik erabiltzen da hainbeste mantentze-lanetan, laguntza-zerbitzuetan eta etengabeko entregetan.

7.4 Zergatik nahastu biak lantalde hibrido batean?

Orain arte esandakoarekin, erantzuna ia agerikoa da. Lantalde hibrido batek **bi erritmo** ditu, oso desberdinak.

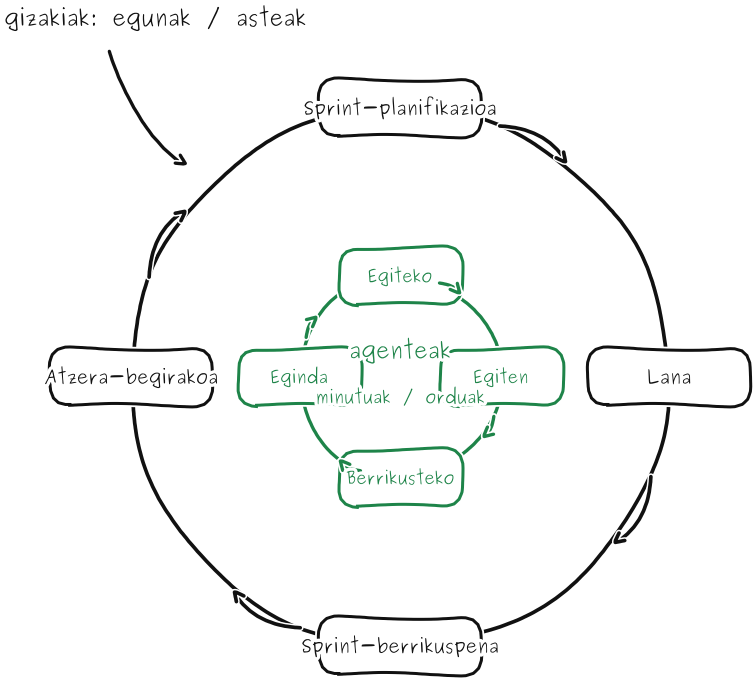
Gizakien erritmoa egunetan eta asteetan neurtzen da: zer eraiki erabaki, zereginak idatzi, egindakoa berrikusi, bezeroarekin edo irakaslearekin hitz egin. Erritmo horrek *Scrum*en egitura behar du. **Sprint-planifikazioak** behartzen gaitu *backlog*a idaztera agenteek ezer ukitu aurretik, eta hori da, hain zuzen, liburu honek irakatsi nahi

duen gaitasun nagusia. **Sprint-berrikuspenak** interesdunak begiztan mantentzen ditu, eta haien iritzia oraindik ere motela eta gizatiarra da, kodea azkar idatzi arren. **Atzera-begirakoan** taldeak zeregin-txantiloiak, zutabe-arauak eta ateak doitzen ditu, eta horrela hobetzen da lantalde hibrido bat. Eta **rolak**: agenteek ez dute produktu-jabearen lana egiten, ez dituzte taldearen oztopoak kentzen, eta ez dira emaitzaren erantzule.

Agenteen erritmoa, aldiz, minututan eta ordutan neurtzen da. Zeregin bat hartu, adar bat sortu, kodea idatzi, testak pasa, *pull request*a ireki. Erritmo horrek *Kanban*en fluxua behar du. Agenteek zereginak **hartu** egiten dituzte *sprint backlog*etik; ez dizkiegu banan-banan eman behar. **Aldi berean lanean dagoen lanaren muga** gizakia muga den zutabeetan jartzen da: berrikuspenean eta giza erabakia behar duten zereginetan. Bestela, agenteek *pull request*ak pilatuko dituzte inork irakurri baino azkarrago, eta hori da lantalde hibridoen lehen ahulezia. **Zutabe-arau esplizituak** dira agenteei eman diezaiekegun gauzarik baliotsuena, zuzen-zuzenean jarraituko dituztelako: noiz dago zeregin bat hartzeko prest, noiz dago eginda, noiz eskalatu behar da gizakiarengana. Eta **ziklo-denbora** da neurri egokia, ez abiadura edo puntuak: zenbat denbora behar du zeregin batek egiteko zutabetik eginda zutabera, eta non itxaroten du. Fluxu horren gakoa da gizakien eta agenteen denborak eta erritmoak sinkronizatu egin behar direla; horregatik, gizakian oinarritzen den lan-automata bat definitzen da, zereginak martxan jartzeko eta gizakien oniritziaren zain geratzeko.

Zergatik ez *Scrum* hutsa? Sprintaren babesak ez du ezer babesten agenteak zeregina minututan bukatzen badu; eguneroko *Scrum*ak ez du zentzurik agenteentzat; eta taldearen abiadura neurtzeak gauza okerra neurtzen du, kodea idaztea ez baita jada lanik astunena. Zergatik ez *Kanban* hutsa? Rolik, planifikaziorik, berrikuspenik eta atzera-begirakorik gabe, ikasleek ikasi behar duten guztia desagertzen da; interesdunen atzeraelikadura-zikloa galtzen da; eta erritmorik gabe, gizakiak itotzen dira etengabe iristen diren berrikuspenetan.

Horregatik, Scrumban agentikoan ***Scrum* inplementaziorako** erabiltzen dugu, metodologia antolatzeko, eta ***Kanban* exekuziorako**, prozesuak egikaritzeko. *Scrum*ek proiektua antolatzen du, gizakien erritmoan; *Kanban*ek zereginak mugitzen ditu, agenteen erritmoan. Lehenak esaten du zer eta noiz; bigarrenak, nola igaro lana sisteman zehar. 7.1 irudian ikus daiteke ideia: kanpoko begizta motela eta barruko begizta azkarra.



7.1. irudia: Scrumban agentikoaren bi erritmoak: *Scrum*en begizta motela gizakientzat, eta *Kanban*en begizta azkarra agenteentzat.

7.5 Scrumban agentikoa, begirada batean

7.1 taulak laburbiltzen du *Scrum*en elementu bakoitza zertan bihurtzen den Scrumban agentikoan. Ia dena mantentzen da; aldatzen dena da nork egiten duen eta zer erritmotan.

Lantaldeak lau rol ditu. **Produktu-jabea** gizakia da, eta lehen bezala erabakitzen du zer eta zer ordenatan. **Garatzaileak** gizakiak dira, baina haien lana aldatu egin da: kodea idatzi beharrean, zereginak idazten dituzte, agenteen lana berrikusten dute eta agenteak erabaki ezin duena erabakitzen dute. **Scrumban-agentea** gainbegirale-agentea da: zereginak esleitzen ditu, agente langileen egoera-txostenak jasotzen ditu, blokeoak antzematen ditu, gizakiarengana eskalatzen du behar denean, eta lana ixten du ateak gainditutakoan. **Agente langileak** kodetzeko agenteak dira, eta bakoitzak zeregin bakarra du esku artean: zeregin bat, adar bat, *pull request* bat.

Metodologia arinetan gertatzen den modura, lanaren unitatea **bukatutako ze-**

***Scrum*en elementua**	**Scrumban agentikoan**
Sprinta	Mantendu, gizakien erritmo gisa; ikasturteko mugarriekin lerrokatuta.
Sprint-planifikazioa	Mantendu; zereginak agenteek egikaritzeko moduan idazten diren unea.
*Sprint backlog*a	Agenteek zereginak hartzen dituzten ilara, aldi berean lanean dagoen lanaren mugekin.
Eguneroko *Scrum*a	Scrumban-agentearen egoera-txostena gizakientzat.
Egindakoaren definizioa	Integrazio jarraituaren eta berrikuspenaren ateak *pull request* bakoitzean.
Scrum zuzendaria	Scrumban-agentea fluxuaren zaintzarako, eta gizakia taldearen prozesurako.
Sprint-berrikuspena	Mantendu; gehikuntza interesdunei erakusten zaie, lehen bezala.
Atzera-begirakoa	Mantendu; txantiloiak, zutabe-arauak eta ateak doitzen dira.

7.1. Taula: *Scrum*en elementuak Scrumban agentikoan.

regina da, 3. kapituluan ikasi genuen moduan idatzia, eta proiektu-taulan bizi da, 5. kapituluan eskuz kudeatu genuen moduan. Zutabeak eta haien arauak dira metodoaren bihotza: Egiteko, Egiten, Berrikusteko eta Eginda, eta horiekin batera Blokeatuta eta gizakia behar duten zereginen zutabea. Zutabe bakoitzak muga bat du eta arau idatziak, eta arau horiek dira agenteei ematen diegun jarraibide nagusia.

Hurrengo ataletan metodoaren mekanika zehaztuko dugu: zeregin baten egoera-makina, egoera batetik bestera nork eta noiz pasatzen duen; eskalatze-protokoloa, agenteak noiz eta nola eskatzen duen gizakiaren laguntza; eta trazabilitatea, zeregin batetik haren adarrera, *pull request*era eta kodera iristeko bidea.

7.6 Zereginaren egoera-makina

7.7 Eskalatze-protokoloa

7.8 Trazabilitatea: zereginetik kodera

8. Ariketak: kaixo, Scrumban agentikoa!

Laburpena (behin-behinekoa): Kapitulu honetan lehen aldiz jarriko dugu agente bat lanean gure fluxuan, ahalik eta modurik txikienean: zeregin bat, agente bat, *pull request* bat. 5. kapituluko taula eta 6. kapituluko agentea hartuko ditugu, eta pausoz pauso egingo dugu bidea: zeregina idatzi onarpen-irizpideekin, agenteari esleitu, agenteak adarra sortu eta kodea idatzi, *pull request*-a ireki, integrazio jarraituak testak exekutatu, gizakiak berrikusi eta bateratu, eta taulako txartela Eginda zutabera iritsi. Ariketa bakoitzean zerbait apurtuko dugu nahita —zeregin lausoa, test faltsua, baimen gehiegi— fluxuak nola erantzuten duen ikusteko. Ariketak doako mailan edo rol-jokoan egin daitezke, ikasleek agentearen papera eginez, tresnarik gabe ere.

8.1 Ariketa ebatziak

8.2 Proposatutako ariketa gehigarriak

9. Ariketak: ekosistema agenteekin

Laburpena (behin-behinekoa): Kapitulu honetan proiektu oso bat garatuko dugu Scrumban agentikoarekin, sprint batzuetan zehar: aurreko eskuliburuko ekosistemaren simulazioa, orain lantalde hibrido batekin. Gizakiek *product backlog*-a idatziko dute eta sprinta planifikatuko dute; agenteek zereginak hartuko dituzte eta *pull request*-ak irekiko dituzte; scrumban-agenteak fluxua zainduko du eta blokeoak eskalatuko ditu; gizakiek berrikusi, bateratu eta sprint-berrikuspena egingo dute. Ariketa bakoitzak egoera erreal bat plantea-tzen du —agente bik fitxategi bera ukitu dute, zeregin bat blokeatuta geratu da, berrikuspenen ilara luzeegia da— eta taldeak erabaki behar du zer egin. Giten historia-grafoa eta proiektu-taula marraztuko ditugu une bakoitzean, au-rreko eskuliburuan egin genuen bezala.

9.1 Ariketa ebatziak

9.2 Proposatutako ariketa gehigarriak

10. DevOps: egiaztapena eta berrikuspena

Laburpena (behin-behinekoa): Kapitulu honetan gizakiaren lanik garrantzitsuena landuko dugu lantalde hibrido batean: egiaztatzea. DevOps ikuspegitik abiatuko gara (Forsgren, Humble eta Kim 2018; Kim et. al. 2016) — integrazio eta entrega jarraitua, dena automatizatu, dena neurtu— eta ikusiko dugu zergatik den are beharrezkoagoa agenteekin: kodea azkar iristen bada, ateak automatikoak izan behar dira. Hiru maila bereiziko ditugu: makinak egiaztatzen duena (testak, analisi estatikoa, konpilazioa, integrazio jarraituaren fluxuak), scrumban-agenteak egiaztatzen duena (zereginaren irizpideak bete diren, mugak errespetatu diren) eta gizakiak bakarrik egiazta dezakeena (diseinua, arkitektura, segurtasuna, zentzua). *Pull request* bat irakurtzen ikasiko dugu: *diff*-a, testak, agentearen azalpena, eta zer galdetu. Eta aurreko eskuliburuko testen ariketa berreskuratuko dugu, orain agenteak idatzitako testak epaitzeko.

10.1 DevOps ikuspegia: integrazio eta entrega jarraitua agenteekin

10.2 Hiru egiaztapen-maila: makina, scrumban-agentea, gizakia

10.3 *Pull request* bat irakurtzen

10.4 Agenteak idatzitako testak epaitzen

11. Ohiko arazoak eta errezetak

Laburpena (behin-behinekoa): Aurreko eskuliburuko azken kapituluaren formatu berean, agenteekin lan egitean sortzen diren arazo ohikoenak bilduko ditugu, bakoitza errezeta batekin: agenteak “eginda” dio baina testek huts egiten dute; agenteak testak aldatu ditu pasa zitezen; bi agentek fitxategi bera aldatu dute eta gatazka sortu da; agenteak zereginaren mugak gainditu ditu eta behar ez zena ukitu du; zeregina lausoegia zen eta agenteak asmatu egin du; berrikuspenen ilara pilatu da eta fluxua gelditu da; agenteak testuingurua galdu du zeregin luze batean; kostua espero baino askoz handiagoa izan da; agenteak sekretu bat edo baimen gehiegi erabili ditu. Errezeta bakoitzak arazoa antzemateko seinaleak, konponbidea eta prebentzioa ditu.

11.1 Agenteak «eginda» dio, baina testek huts egiten dute

11.2 Agenteak testak aldatu ditu pasa zitezen

11.3 Bi agentek fitxategi bera aldatu dute

11.4 Agenteak zereginaren mugak gainditu ditu

11.5 Zeregina lausoegia zen eta agenteak asmatu egin du

11.6 Berrikuspenen ilara pilatu da eta fluxua gelditu da

11.7 Agenteak testuingurua galdu du zeregin luze batean

11.8 Kostua espero baino askoz handiagoa izan da

11.9 Agenteak sekretu bat edo baimen gehiegi erabili ditu

12. Eranskina: txantiloiak eta konfigurazioa

Laburpena (behin-behinekoa): Eranskinak liburuan erabilitako txantiloiak eta konfigurazioak biltzen ditu, kopiatzeko prest: zeregin-txantiloia (*issue*-txantiloia) eta *pull request*-txantiloia; proiektu-taularen zutabeak, mugak eta arauak; adarren babes-arauak; integrazio jarraituaren *workflow* bat (liburu hau bera konpilatzen duena, adibide gisa); agenteen testuinguru-fitxategi baten eredua (proiektuaren arauak agentearentzat); eta scrumban-agentearen jarraibideak. Tresna zehatzei lotutako bertsioak liburuaren biltegi lagunean daude, eguneratuta; hemen laburpena eta estekak baino ez.

12.1 Zeregin- eta *pull request*-txantiloiak

12.2 Proiektu-taularen zutabeak, mugak eta arauak

12.3 Lerro nagusiaren babes-arauak

12.4 Integrazio jarraituaren *workflow*-a

12.5 Agenteen testuinguru-fitxategia

12.6 Scrumban-agentearen jarraibideak

Bibliografia

Anderson, David J. (2010). *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press.

Bandara, Eranga et. al. (2025). *Agentsway – Software Development Methodology for AI Agents-based Teams*. arXiv:2510.23664. Agenteak taldekide osoak diren lantaldeetarako metodologia; lan erlazionatua.

Beck, Kent (2002). *Test-Driven Development: By Example*. Addison-Wesley.

Belbin, R. Meredith (2010). *Team Roles at Work*. 2. edizioa. Butterworth-Heinemann.

Chacon, Scott eta Straub, Ben (2014). *Pro Git*. 2. edizioa. Apress.

Cohn, Mike (2004). *User Stories Applied: For Agile Software Development*. Addison-Wesley.

Forsgren, Nicole, Humble, Jez eta Kim, Gene (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press.

Fowler, Martin (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3. edizioa. Addison-Wesley.

Freeman, Steve eta Pryce, Nat (2009). *Growing Object-Oriented Software, Guided by Tests*. Addison-Wesley.

Humble, Jez eta Farley, David (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.

Kim, Gene et. al. (2016). *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press.

Ladas, Corey (2009). *Scrumban: Essays on Kanban Systems for Lean Software Development*. Modus Cooperandi Press.

Lopez-Gazpio, Inigo eta Soroa Etxabe, Aitor (2024). *Git Bertsioak Kontrolatzeko Sistemarako eskuliburua*. Bilbo: Udako Euskal Unibertsitatea (UEU) eta Euskal Herriko Unibertsitatea (UPV/EHU). ISBN: 978-84-8438-894-4.

Object Management Group (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. `https://www.omg.org/spec/UML/2.5.1`.

Ohno, Taiichi (1988). *Toyota Production System: Beyond Large-Scale Production*. Productivity Press.

Rubin, Kenneth S. (2012). *Essential Scrum: A Practical Guide to the Most Popular Agile Process*. Addison-Wesley.

Schwaber, Ken eta Sutherland, Jeff (2020). *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. `https://scrumguides.org`.

Winters, Titus, Manshreck, Tom eta Wright, Hyrum (2020). *Software Engineering at Google: Lessons Learned from Programming Over Time*. O'Reilly Media.